Things to learn after 15th january:

https://www.cockroachlabs.com/blog/what-is-a-uuid/

https://en.wikipedia.org/wiki/Snowflake_ID

https://www.postgresql.org/docs/current/datatype-enum.html

https://www.geeksforgeeks.org/dbms/difference-between-soft-delete-and-hard-delete/

https://developer.mozilla.org/en-US/docs/Web/API/URL/searchParams

https://medium.com/better-programming/understanding-the-offset-and-cursor-pagination-8ddc54d10d98

https://www.geeksforgeeks.org/system-design/eventual-consistency-in-distributive-systems-learn-system-design/

PRD (product requirement document)

- Contacts
- End to end encryption
- Read Receipts
- Attachments
- Disappearing msg
- last seen
- Edit Messages
- Push Notifications
- Room or groups
- typing toggle
- Block / Report Users
- Email verification
- Chat Lock
- delete messages
- Sharing live location
- Message Scheduling
- Auth
- Editing Messages
- Pagination

TRD (technical requirement document)

- Auth
    - Login
        - Email

        - Password
    - Register
        - Email
        - Password
        - Conforming password
- Search the user
    - Db query, return the user
- Send Messages
    - API -> text message to DB
    - Fire a Socket singal
    -
    - Send Attachements (voicenote, document ….)
        - API -> upload your blob into s3
        - Get its presigned url
        - Put into your db
        - Fire socket command
- Create Group Chat
    - Search the users to add in the group
    - Create the group
    - Messages should be delivered to everyone
    - Search user (API call -> list of users which match your search parameter)
    - Select one of the user
    - API call to create a group chat with all the selected users

- End to end encryption
    - Public key
    - Private key
- Read Receipts
    - Group chat (100 people)
    - Hi
    - For every message, you store if that message was seen by a user or not
- Last seen
    - User property
    - Whenever user logs in update it (online)
    - Whenever user logs out (store time)
    - When user sends a message, we will update the last seen
    - Whenever user sees a message, we will update the last seen
- Typing
    - Starts typing, frontend send socket commands over the chat
- Delete Messages
    - isActive field in the DB, (boolean) . false

DB schema

- Keys
    - UserId
    - Public Key

- User
    - Id (primary_key, notnull, varchar) uuids
    - Name (varchar, not null)
    - Email (varchat, not null)
    - Password (encrypted) (varchar, not null)
    - Created_at (timestamp)
    - Updated_at (timestamp)
    - Last_seen (timestamp)
    - Private Key (varchar)

- Chat
    - id (primary_key, notnull, varchar) uuids
    - admin_user ([]) list of users who are admin for that chat
    - Chat_users ([]) list of users that are in the chat
    - Created_at
    - Update_at
    - chatStatus (enum (LOCKED, ACTIVE))

- UserChatMetadata
    - ChatId
    - UserId
    - ChatMode
    - Status (enum (MUTED, ARCHIVED))

- Message
    - Id
    - text
    - Blob_location
    - File_type
    - Created_at
    - Updated_at
    - Chat_id'
    - isActive (boolean , default true)
    - isEdited (boolean, default false)

- UserMessageMetadata
    - Messageid
    - UserId
    - messageStatus (READ, UNREAD)

APIS

- PUT -> replacing the entire object with a new one

- PATCH -> User object, change only or two properties of that object
- DELETE -> if you are trying to delete some data (soft delete)
- GET -> get data from server, then use GET (there is no requestBody in GET)
- POST -> send some data to you server

API design

User

- Register User
    - POST (/user/register)
        - Sending (username, email, password)
        - Send to you backend
- Login User
    - POST (/user/login)
        - Sending (email, password)
- Search User
    - GET (/user/search)
        - Eg : /user/search?email=example@gmail.com&name=example

Chat

- Create Chat
    - POST (/chat/create)
        - Sending (user[], userId)
- Block Chat
    - POST (/chat/block)
        - Sending (chatId, userId)
        - If the user is admin in a groupChat, it is blocked
        - If one-on-one chat, blocked
- MakeAdmin
    - PATCH (/chat/makeAdmin)
    - Sending (new_admin_id, user_id)
- RemoveFromChat
    - PATCH (/chat/remove)
    - Sending (chatId, userId, user_to_remove)
- LeaveChat
    - PATCH (/chat/leave)
    - Sending (chatId, userId)
- GetMessages
    - GET (/chat/messages)
    - Eg : /chat/messages?chatId=uuid&limit=50&offset=100

Message

- Send Message
    - POST (/message)
    - Sending (userid, chatid, text)
- Edit Message
    -
- Delete message
- Send Attachment